

Lab 5 – DNN on MNIST (Group 0)

Fixed-Point Optimizations and Embedded Inference

AES Course

February 25, 2026

Contents

1	Introduction	2
2	Network Architecture	2
3	Fixed-Point Formats (Q)	4
3.1	Formats Used	4
3.2	Conversion Between Formats	4
4	Implemented Optimizations	4
4.1	MODE 0 – Baseline Configuration	4
4.2	MODE 1 – TIP1 (Image format optimization)	4
4.3	MODE 2 – TIP1 + TIP2 (Image + weight format optimization)	5
4.4	Comparison with Optimizations	5
5	Software Implementation	5
5.1	Code Structure	5
5.1.1	Configuration and Constants	5
5.1.2	Global Variables	6
5.1.3	Main	6
5.1.4	UART	6
5.1.5	Fully-Connected Layers	6
5.1.6	ReLU and Post-Processing	6
5.1.7	DNN Inference	6
5.1.8	Offline Test	7
5.1.9	UART Loop	7
6	Experimental Results	7
6.1	Validation on Embedded Images	7
6.2	UART Test	8
7	Conclusions	10

1 Introduction

This report describes the implementation of a fully-connected neural network for MNIST classification, optimized for execution on an embedded platform (Zynq). The network was developed for **Group 0** and applies the optimizations described in the document “Tips and tricks for optimization”.

The main goal is to reduce memory and transmission time through the use of fixed-point arithmetic (Q8.8, Q0.8, Q1.7 formats) while maintaining acceptable accuracy.

2 Network Architecture

The implemented network has topology **784** $\rightarrow$ **64** $\rightarrow$ **10**:

- **Input**: 28×28 grayscale MNIST image (784 pixels)
- **FC0**: fully-connected layer 784→64
- **ReLU**: nonlinear activation
- **FC1**: fully-connected layer 64→10 (output)

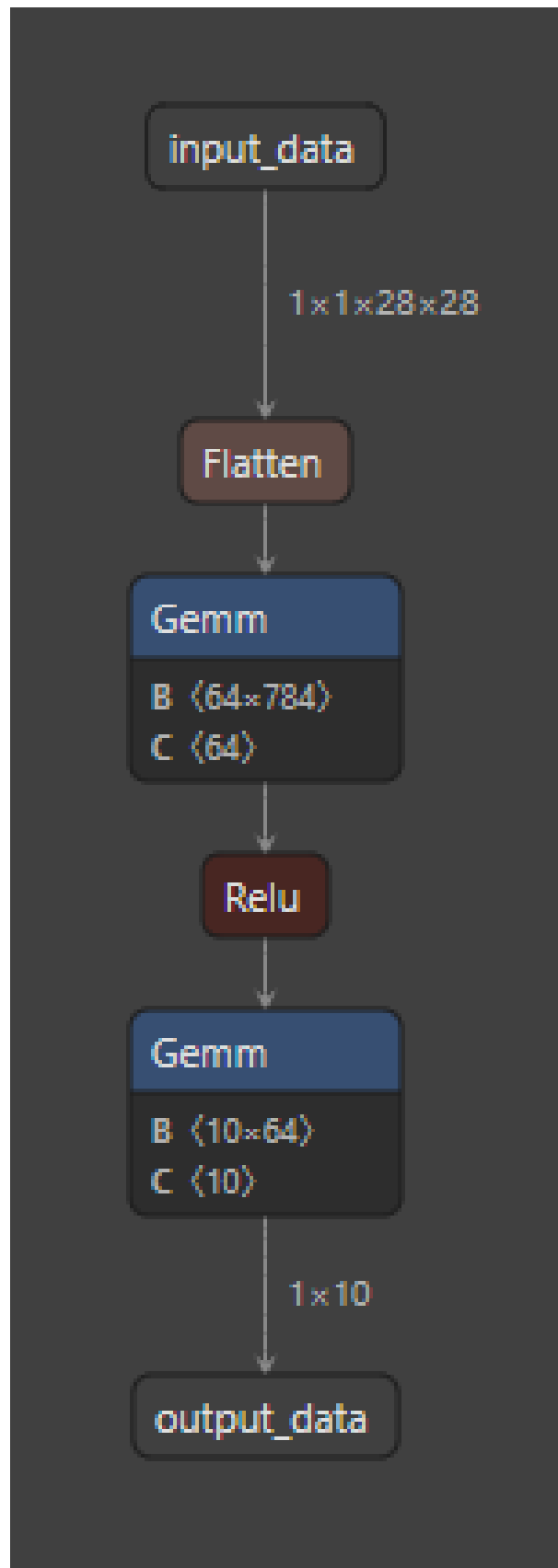


Figure 1: Neural network architecture (conceptual diagram). Group 0 implementation uses $784 \rightarrow 64 \rightarrow 10$.

3 Fixed-Point Formats (Q)

As stated in “Tips and tricks for optimization” 1, the chosen format for weights and activations is **Q8.8**. Fixed-point arithmetic is essential for embedded systems. The **Qm.n** notation denotes:

- **m**: bits for the integer part (including sign)
- **n**: bits for the fractional part
- Real value: $\frac{\text{represented_value}}{2^n}$

3.1 Formats Used

Format	Bits	C type	Use	Range
Q8.8	16	short	Images, weights (baseline)	$\approx -128 - +128$
Q0.8	8	unsigned char	Images via UART (TIP1)	$0 - \approx 1$
Q1.7	8	signed char	Weights (TIP2)	$\approx -1 - +1$

Table 1: Fixed-point formats used in the project

3.2 Conversion Between Formats

- **Q8.8** $\rightarrow$ **Q0.8**: $q08 = q88 \gg 8$ (UART bandwidth reduction)
- **Q8.8** $\rightarrow$ **Q1.7**: $q17 = \text{clamp}(q88 \gg 1, -128, 127)$ (weight memory reduction)
- **Q1.7** $\rightarrow$ **Q8.8** (in C): $q88 = (\text{short})q17 \ll 1$

4 Implemented Optimizations

4.1 MODE 0 – Baseline Configuration

MODE 0 is the reference configuration for Group 0, with no optimizations. All data use Q8.8 format (16 bit):

- **Images**: 2 bytes per pixel $\Rightarrow$ 1568 bytes per image via UART
- **Weights**: 2 bytes per weight $\Rightarrow$ ~ 100 KB total ($50176 + 640 + 128 + 20$)
- **Precision**: maximum, no additional quantization

This configuration serves as a reference to compare accuracy and performance of the optimized versions.

4.2 MODE 1 – TIP1 (Image format optimization)

As stated in “Tips and tricks for optimization”, inspecting the image data with a hex editor shows that the **most significant byte of each pixel is always 0x00**: MNIST pixels are in $[0, 1]$ and are positive, so in Q8.8 the high byte carries no useful information. **TIP1** exploits this by using a **Q0.8** representation for the image over the link: 1 byte per pixel (8 fractional bits, no integer part) instead of 2 bytes (Q8.8).

Rationale. Sending 784 bytes per image instead of 1568 cuts UART transfer time in half. This is the main bottleneck when images are sent repeatedly (e.g. from a host PC or camera).

Implementation. On the host/sender side, images are converted to Q0.8 (e.g. `q08 = q88 » 8`) before transmission. On the embedded board, `readfromUART()` reads a single byte per pixel and reconstructs Q8.8 with `(short)px « 8`, so the rest of the pipeline (FC layers, ReLU, argmax) is unchanged. Weights and biases stay in Q8.8; only the image input format and UART protocol change.

4.3 MODE 2 – TIP1 + TIP2 (Image + weight format optimization)

Inspecting the weight and bias files with a hex editor shows that the **most significant byte of each 16-bit value is always 0x00 or 0xFF**: the magnitude is in the fractional part, and the integer part effectively encodes only the sign. So weights and biases can be represented in **Q1.7** (1 sign bit, 7 fractional bits) with minimal impact on accuracy.

Rationale. Storing weights and biases in Q1.7 (`signed char`) halves memory usage (1 byte per parameter instead of 2). Saving time when sending weights over UART is a minor gain, since DNN parameters are typically sent once at startup; the main benefit is **reduced on-board memory occupation**.

Implementation. Weight and bias arrays are stored in Q1.7 in `weights_group0.h` (e.g. `weights0_q17`, `bias0_q17`). In `FC_forward_q17()`, each 8-bit weight/bias is scaled back for the MAC: `(short)weights[i] « 1` and `(short)bias[h] « 1` align Q1.7 to the internal Q8.8 scale so that the existing fixed-point MAC logic (with `» qf`) remains correct. No change to the image path: MODE 2 still uses TIP1 for the image (784 bytes UART). Accuracy stays comparable to the baseline because the useful information in the weights is in the fractional part, which Q1.7 preserves.

4.4 Comparison with Optimizations

Parameter	MODE 0	MODE 1 (TIP1)	MODE 2 (TIP1+TIP2)
UART bytes/image	1568	784	784
Weight memory	~100 KB	~100 KB	~50 KB
Image format	Q8.8	Q0.8→Q8.8	Q0.8→Q8.8
Weight format	Q8.8	Q8.8	Q1.7→Q8.8
Reduction vs baseline	–	50% UART	50% UART + 50% weights

Table 2: Comparison of MODE 0 (baseline) with optimizations

5 Software Implementation

5.1 Code Structure

The main file `mnist_group0.c` implements a two-layer fully-connected DNN for MNIST classification (784→64→10) on an embedded platform (Zynq), with three modes selectable via `MODE` (0=baseline, 1=TIP1, 2=TIP1+TIP2). Below is the general code structure.

5.1.1 Configuration and Constants

- **MODE:** 0 = baseline (Q8.8 images via UART, normal weights); 1 = TIP1 (Q0.8 images, 784 bytes UART); 2 = TIP1+TIP2 (Q0.8 + Q1.7 weights).
- **Dimensions:** `IMG_SIZE=784`, `N_BIAS0/1`, `N_WEIGHTS0/1` for the two layers.
- **DATA:** `short int` type for activations and weights.

- Macro `FIXED2FLOAT`: conversion from fixed-point to float for the final result.

5.1.2 Global Variables

- Weights and bias: in MODE 2, `signed char` arrays (Q1.7) from `weights_group0.h` are used; otherwise `DATA` (Q8.8).
- `imagine[10][IMG_SIZE]`: buffer for the 10 offline test images.
- `Uart_1_PS`: UART instance for image reception.

5.1.3 Main

- Initializes platform and UART; prints banner with active mode.
- Runs *offline test* on the 10 images in `test_images.h`.
- Enters the *UART loop* that receives images and runs inference in a continuous cycle.

5.1.4 UART

- `uart_inbyte()`: reads one byte (blocking).
- `readfromUART()`: in MODE 0 reads 2 bytes (Q8.8, LSB-first); in MODE 1/2 reads 1 byte per pixel and interprets it as Q0.8 (`(short)px << 8`).
- `uart_init()`: configures UART at 115200 baud.
- `uart_receive_image()`: reads `IMG_SIZE` values and writes them to `image[]`.

5.1.5 Fully-Connected Layers

- `FC_forward()`: for each output neuron accumulates `bias[h]<<qf` and the MAC with `input` and `weights`; result in fixed-point with `>> qf`.
- `FC_forward_q17()` (MODE 2 only): same logic with `signed char` weights/bias (Q1.7); bias and weights are scaled with `<< 1` to align with internal format (Q8.8).

5.1.6 ReLU and Post-Processing

- `relu_forward()`: `output[i] = max(0, input[i])`.
- `resultsProcessing()`: converts the last 10 logits from fixed to float, finds the index of the maximum and returns it as the predicted class (0–9).

5.1.7 DNN Inference

The function `dnn_inference(image)`:

1. FC0: 784→64 (uses `FC_forward` or `FC_forward_q17` depending on MODE).
2. ReLU on the 64 outputs → `in1`.
3. FC1: 64→10.
4. `resultsProcessing(out1, 10)` → predicted class index.

5.1.8 Offline Test

Copies the 10 images from `imm_test_0-imm_test_9` (in `test_images.h`) into `immagine[] []`; for each image calls `dnn_inference(immagine[i])` and prints `expected=i, predicted=pred`.

5.1.9 UART Loop

Message indicating the expected format (1568 bytes in MODE 0, 784 bytes in MODE 1/2). In the loop: records time `t0`, receives the image (`t1`), runs `dnn_inference` (`t2`), prints the prediction and `usleep`, then `t3`. Prints times in μ s: RX (reception), DNN (inference), PRINT (print + sleep), total.

Data flow.

UART (784 or 1568 bytes) $\rightarrow$ `image[]` $\rightarrow$ FC0(784 $\rightarrow$ 64) $\rightarrow$ ReLU $\rightarrow$ FC1(64 $\rightarrow$ 10) $\rightarrow$ argmax $\rightarrow$ class 0–9

6 Experimental Results

6.1 Validation on Embedded Images

Validation uses 10 reference images (digits 0–9) embedded in the firmware. For each image, `expected` and `predicted` are printed.

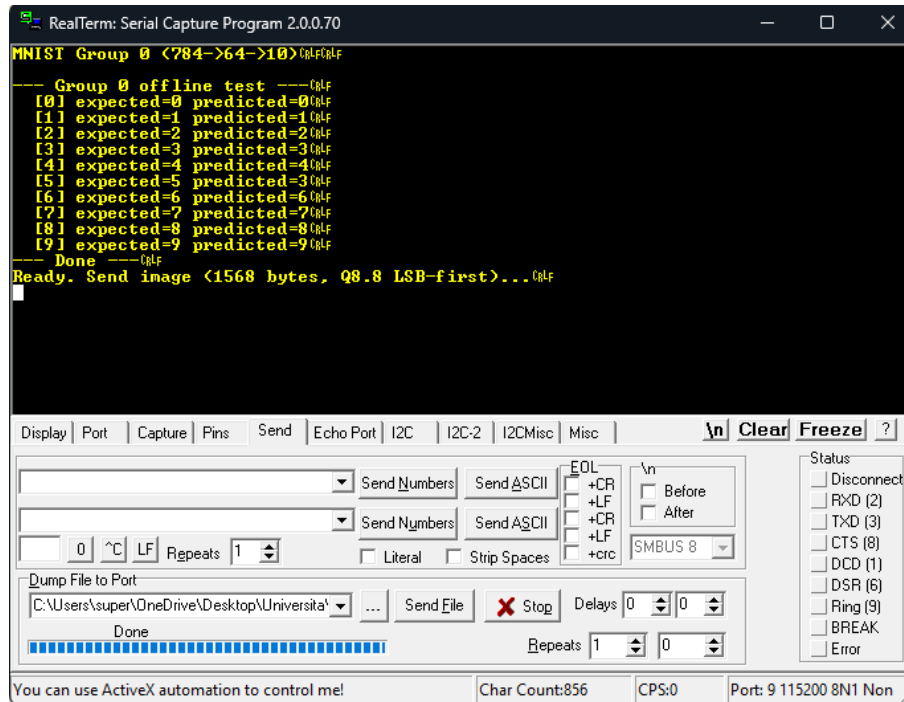


Figure 2: Validation – MODE 0 (baseline Q8.8)

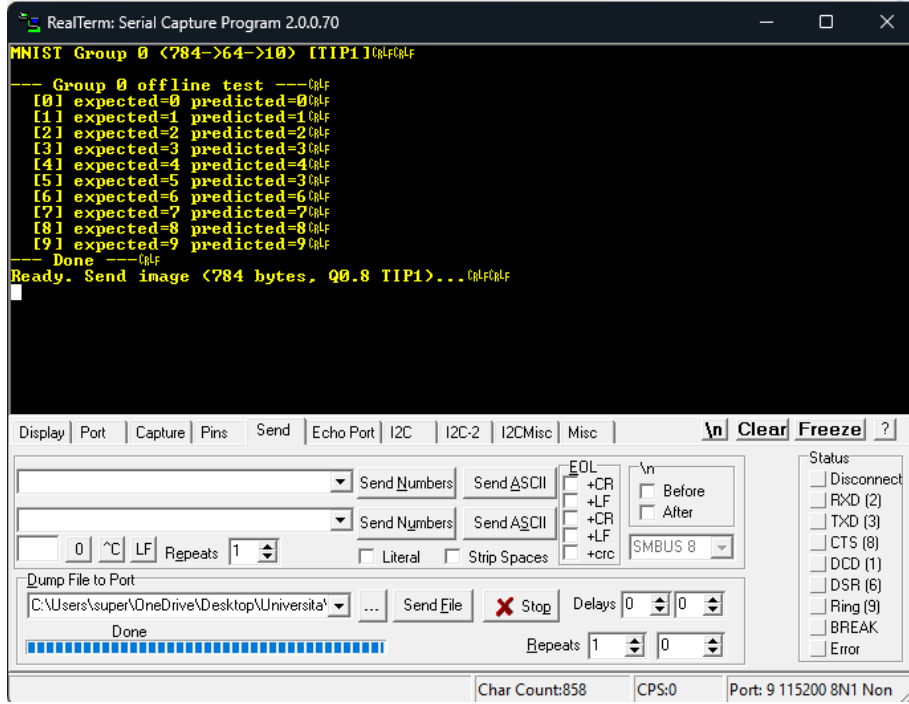


Figure 3: Validation – MODE 1 (TIP1, Q0.8 images)

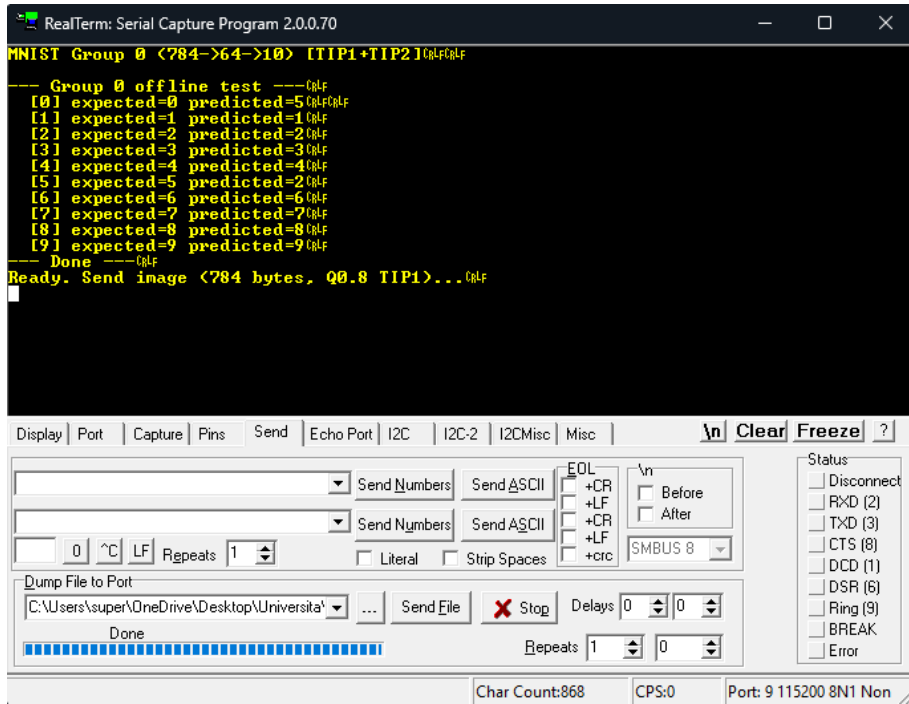


Figure 4: Validation – MODE 2 (TIP1+TIP2, Q1.7 weights)

6.2 UART Test

With images sent from the PC (Q0.8 format for TIP1/TIP2, Q8.8 for baseline), the board responds with the prediction and measured times (RX, DNN, PRINT).

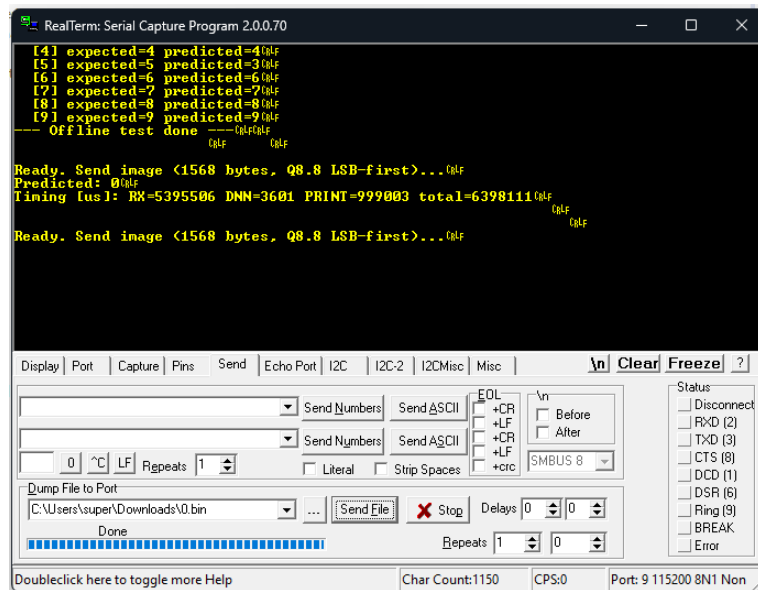


Figure 5: Example of UART prediction – digit 0 image

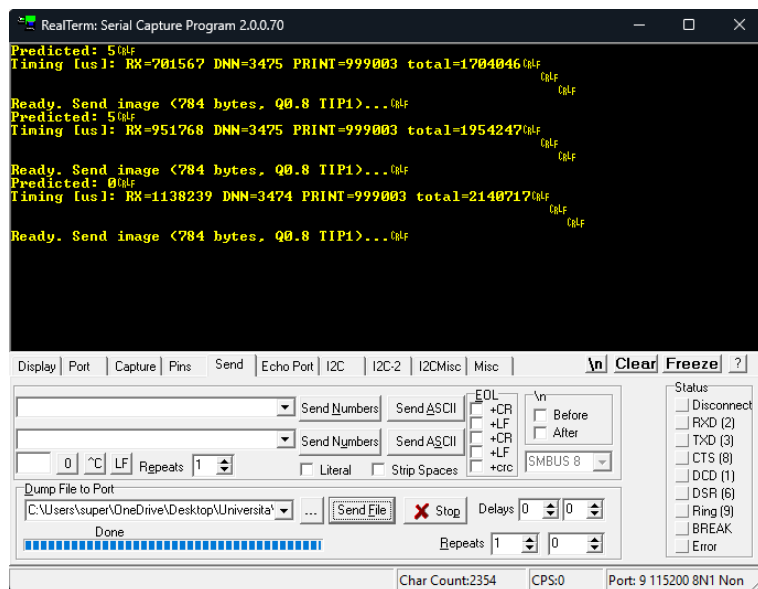


Figure 6: UART prediction with TIP1

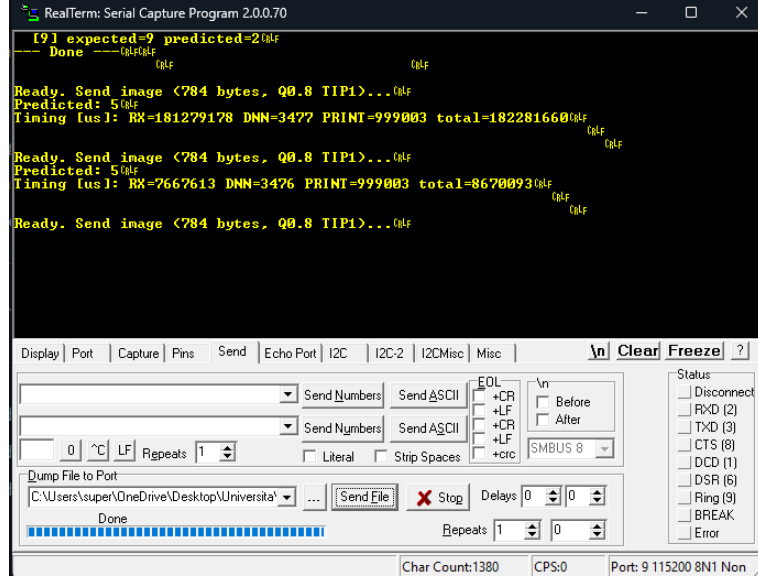


Figure 7: UART prediction with TIP1+TIP2

7 Conclusions

The implementation shows that TIP1 and TIP2 optimizations allow to:

- Reduce UART traffic by 50% (784 vs 1568 bytes per image)
- Reduce weight memory by 50% (Q1.7 vs Q8.8)
- Maintain accuracy comparable to the baseline

Fixing the overflow in image initialization was essential to obtain correct predictions in all modes.

References

1. Tips and tricks for optimization (PDF), course material – fixed-point formats (Q8.8, Q0.8, Q1.7), Tip 1 (image Q0.8 over UART), Tip 2 (weights/bias Q1.7, memory and UART).